

AdaptPrep: AI-Powered Adaptive Learning **Platform for JEE & NEET Preparation**

Submitted **Capstone Project** in partial fulfilment of
the requirements of the degree

B. Tech in Computer Engineering

Second Year

Subject : Skill Based Lab-Advanced Web Technologies (SBL-AWT)

By

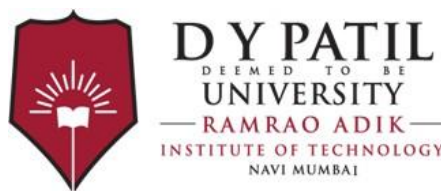
25CE1116: Purval Singh

25CE1124: Atharva Dorugade

25CE1120: Nishant Sunil Zemse

Supervisor

Subject Faculty Name



Department of Computer Engineering
Ramrao Adik Institute of Technology,
Sector 7, Nerul, Navi Mumbai
(Under the ambit of D. Y. Patil Deemed to be University)
November 2026

Abstract

AdaptPrep is a full-stack web platform that helps students prepare for India's JEE Main and NEET UG entrance examinations through adaptive, AI-assisted practice. Instead of serving the same static question papers to every learner, the system records each student's performance topic by topic and uses a weighted-sampling algorithm to assemble "AI Adaptive" tests that focus on weak and unexplored topics, prefer unseen questions and tune difficulty to the student's current level. The platform supports three roles: students take timed tests with exam-style marking (+4/-1), review detailed solutions, track analytics such as accuracy, streaks and an exam-readiness score, follow an auto-generated study plan and chat with an AI tutor; teachers create classes, author questions and assign tests; and administrators manage users, classes and audit logs. The frontend is a React single-page application built with Vite, and the backend is a Node.js/Express REST API secured with JWT authentication, bcrypt password hashing, rate limiting and role-based access control. AI features run on a built-in local engine with optional Google Gemini integration, so the application works fully offline and degrades gracefully. The application is deployed on Vercel, with the API running as a serverless function and data persisted to Upstash Redis.

Content

1. Aim : Problem statement
2. Softwares / Tools used
3. Proposed System
 - Working of project
 - Block diagram / Flow diagram
 - Algorithm / Flowchart
 - Code snippets
4. Results and Discussion : Screenshots of results
5. Conclusion & future scope
6. References

1. Aim : Problem Statement

Aim: To design and develop a secure, role-based web application that personalises JEE and NEET preparation by adapting practice tests to each student's strengths and weaknesses and by providing AI-driven guidance, analytics and classroom tools.

Problem statement: Lakhs of students prepare for JEE and NEET every year, mostly using fixed question banks and mock tests that are identical for every learner. A student who is already strong in Mechanics keeps receiving Mechanics questions, while weak areas such as Organic Chemistry get too little practice. Students rarely get timely feedback on where they are losing marks, and teachers have no simple way to assign targeted work and monitor a class. Existing platforms are often paid, need constant internet access to an external AI service, or offer little insight beyond a raw score.

The project therefore addresses the following objectives:

- Generate adaptive tests that prioritise each student's weak and unexplored topics.
- Replicate real exam conditions: timers, sections and negative marking (+4 correct, -1 incorrect).
- Provide analytics (accuracy per subject/topic, score trends, streaks and a readiness score) and an automatic weekly study plan.
- Offer an AI tutor and AI question generation that work even without an external API key.
- Give teachers classes, assignments and question authoring, and give administrators user management and audit logs.
- Keep the platform secure (hashed passwords, JWT sessions, role checks, rate limiting) and deployable to the cloud.

2. Softwares / Tools Used

Category	Software / Tool
Frontend	React 18, React Router 6, Vite 5, Recharts (charts), Lucide React (icons), HTML5, CSS3
Backend	Node.js (v18+), Express 4 REST API
Security	JSON Web Tokens (jsonwebtoken), bcryptjs password hashing, CORS, security headers, in-memory rate limiting
Database	Custom document store: JSON file locally, Upstash Redis (REST API) in production
Artificial Intelligence	Built-in "AdaptPrep Local AI" engine (retrieval + templated generation); optional Google Gemini 2.5 Flash API
Testing	Node.js built-in test runner (node:test), Playwright (UI screenshots)
Deployment & DevOps	Vercel (static hosting + serverless functions), Git & GitHub
Editor / OS	Visual Studio Code, Windows / Linux, Google Chrome

3. Proposed System

3.1 Working of the Project

AdaptPrep follows a client–server architecture. The React single-page application (SPA) runs in the browser and talks to the Express backend through a JSON REST API under `/api`. After signing up or logging in, the server issues a signed JWT that the client stores and sends in the **Authorization** header of every request. Middleware verifies the token, loads the user and enforces role-based access (student, teacher or admin).

Student flow: the dashboard shows readiness, accuracy, streak and score trends. The student starts a test in one of several modes: *practice* (fixed test sets), *AI adaptive*, *full mock*, *custom*, *revision* (bookmarked or wrong questions) or *assignment*. The server builds the question set, stores an attempt with a deadline and saves progress as the student answers. On submission (or automatically when the timer expires) each answer is graded with +4/–1 marking, and per-question results with explanations are stored. The analytics service aggregates all attempts into subject/topic accuracy, weak and strong topics and a readiness score (60% accuracy + 40% syllabus coverage), which feed the study plan and the AI tutor.

Teacher flow: teachers create classes with join codes, write or AI-generate questions, publish assignments with due dates and view each student's progress. **Admin flow:** administrators manage users (suspend, change role, reset), view all classes, read the audit log and check system status.

AI layer: an orchestrator sends tutor chats and question generation to Google Gemini when an API key is configured and always falls back to the local engine, which answers from a curated knowledge base and the student's own analytics. The AI therefore never breaks and costs nothing to run.

3.2 Block Diagram

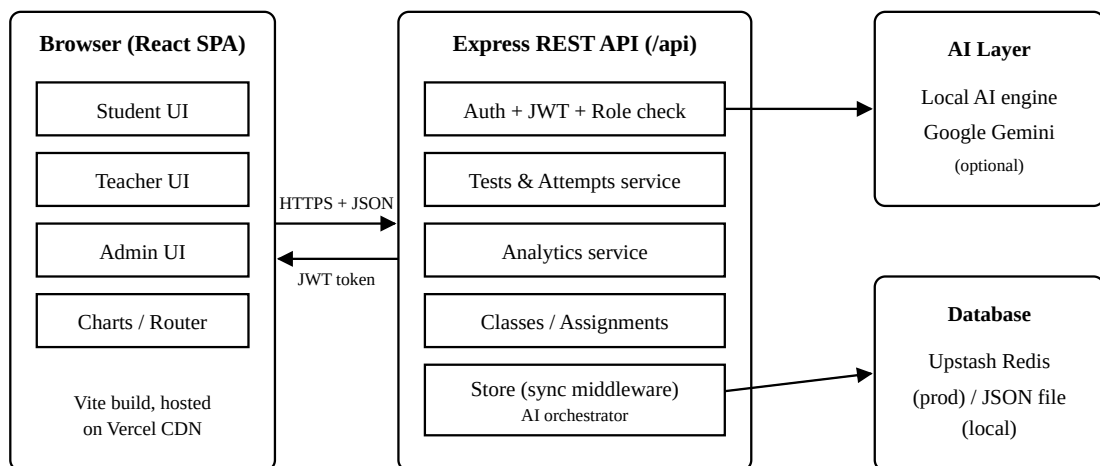


Figure 3.1: Block diagram of AdaptPrep

3.3 Flow Diagram (Test-taking flow)

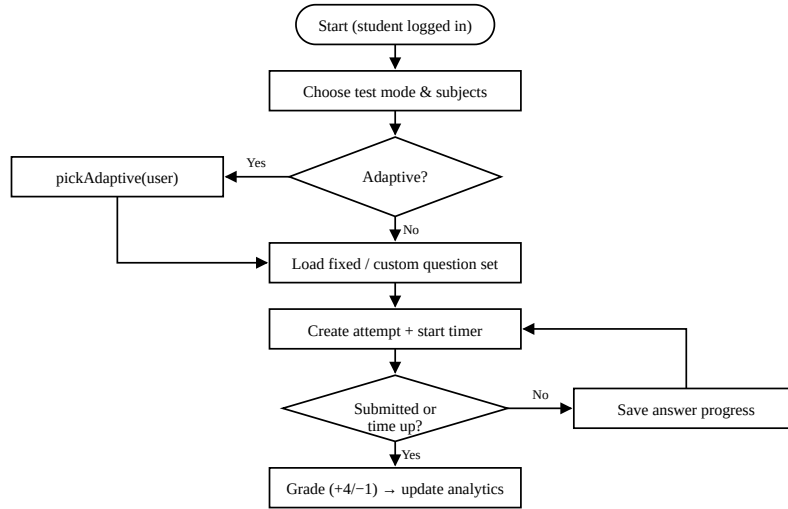


Figure 3.2: Flow diagram of a test attempt

3.4 Algorithm: Adaptive Question Selection

The adaptive test uses **weighted random sampling without replacement** (the Efraimidis–Spirakis method). Each candidate question receives a weight, and the questions with the largest keys $u^{1/w}$ (where u is uniform on $(0, 1)$) are selected. This favours high-weight questions while keeping every test different.

1. Collect all submitted attempts of the student and compute accuracy for every (subject, topic) pair; record the set of questions already seen.
2. Build the pool of active questions for the student's exam (JEE/NEET) and the chosen subjects.
3. For each question q in the pool, with topic accuracy acc :
 - If the topic is unexplored, $w = 2.2$; otherwise $w = 1 + 3 \cdot (1 - acc)$ (weaker topic $\rightarrow$ higher weight).
 - If q has never been seen, $w = w \times 1.8$.
 - If $acc > 0.75$ and q is easy, $w = w \times 0.4$; if $acc < 0.4$ and q is hard, $w = w \times 0.5$.
 - $key(q) = \text{random}()^{1/w}$
4. Sort the questions by key in descending order and return the first N (default 15).
5. On submission, $score = 4 \times \text{correct} - 1 \times \text{incorrect}$; $\text{percent} = \max(score, 0) / (4 \times N) \times 100$.
6. $\text{Readiness} = \text{round}(0.6 \times \text{overall accuracy} + 0.4 \times \text{syllabus coverage})$.

3.5 Code Snippets

(a) Adaptive selection (backend/services/attempts.js)

```
const pickAdaptive = (user, subjects, count) => {
  const { map, seen } = topicAccuracy(user.id);
  const pool = activeQuestions((q) => q.examType === user.examType &&
subjects.includes(q.subject));
  if (!pool.length) throw badRequest('No questions are available for this selection
yet.');
```

```
  // Weight: weak and unexplored topics first, unseen questions preferred,
  // difficulty nudged towards the student's current level.
  const scored = pool.map((q) => {
    const stats = map.get(`${q.subject}:${q.topic}`);
    const acc = stats ? stats.correct / stats.attempted : null;
    let weight = acc === null ? 2.2 : 1 + (1 - acc) * 3;
    if (!seen.has(q.id)) weight *= 1.8;
    if (acc !== null && acc > 0.75 && q.difficulty === 'easy') weight *= 0.4;
    if (acc !== null && acc < 0.4 && q.difficulty === 'hard') weight *= 0.5;
    return { q, key: Math.random() ** (1 / weight) };
  });
  scored.sort((a, b) => b.key - a.key);
  return scored.slice(0, count).map((s) => s.q);
};
```

(b) JWT authentication middleware (backend/middleware/auth.js)

```
const signToken = (user) => jwt.sign(
  { sub: user.id, role: user.role, tv: user.tokenVersion || 0 },
  getJwtSecret(),
  { expiresIn: process.env.JWT_EXPIRE || '7d' }
);

const auth = (req, res, next) => {
  const header = req.header('Authorization') || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return next(unauthorized());
  let payload;
  try {
    payload = jwt.verify(token, getJwtSecret());
  } catch (err) {
    return next(unauthorized('Your session has expired. Please sign in again.));
  }
  const user = getStore().byId('users', payload.sub);
  if (!user || (user.tokenVersion || 0) !== payload.tv) {
    return next(unauthorized('Your session is no longer valid. Please sign in
again.));
  }
  ...
};
```

(c) REST API routing (backend/app.js)

```
app.use('/api/public', require('./routes/public'));
app.use('/api/auth', require('./routes/auth'));
app.use('/api/me', auth, require('./routes/me'));
app.use('/api/tests', auth, require('./routes/tests'));
app.use('/api/analytics', auth, require('./routes/analytics'));
app.use('/api/classes', auth, require('./routes/classes'));
app.use('/api/ai', auth, require('./routes/ai'));
app.use('/api/admin', auth, requireRole('admin'), require('./routes/admin'));
```

(d) Frontend API client (frontend/src/api/client.js)

```
const BASE = import.meta.env.VITE_API_URL || '/api';
const TOKEN_KEY = 'adaptprep-token';
export const tokenStore = {
  get: () => { try { return localStorage.getItem(TOKEN_KEY); } catch { return null; } },
  set: (t) => { try { localStorage.setItem(TOKEN_KEY, t); } catch { /* private mode */ } },
  clear: () => { try { localStorage.removeItem(TOKEN_KEY); } catch { /* private mode */ } }
};
```


4. Results and Discussion

The application was tested end-to-end with an automated suite of 17 API tests (authentication, validation, role checks, test attempts, grading, and database persistence), all of which pass. The screenshots below were captured from the running application using the seeded demo workspace.

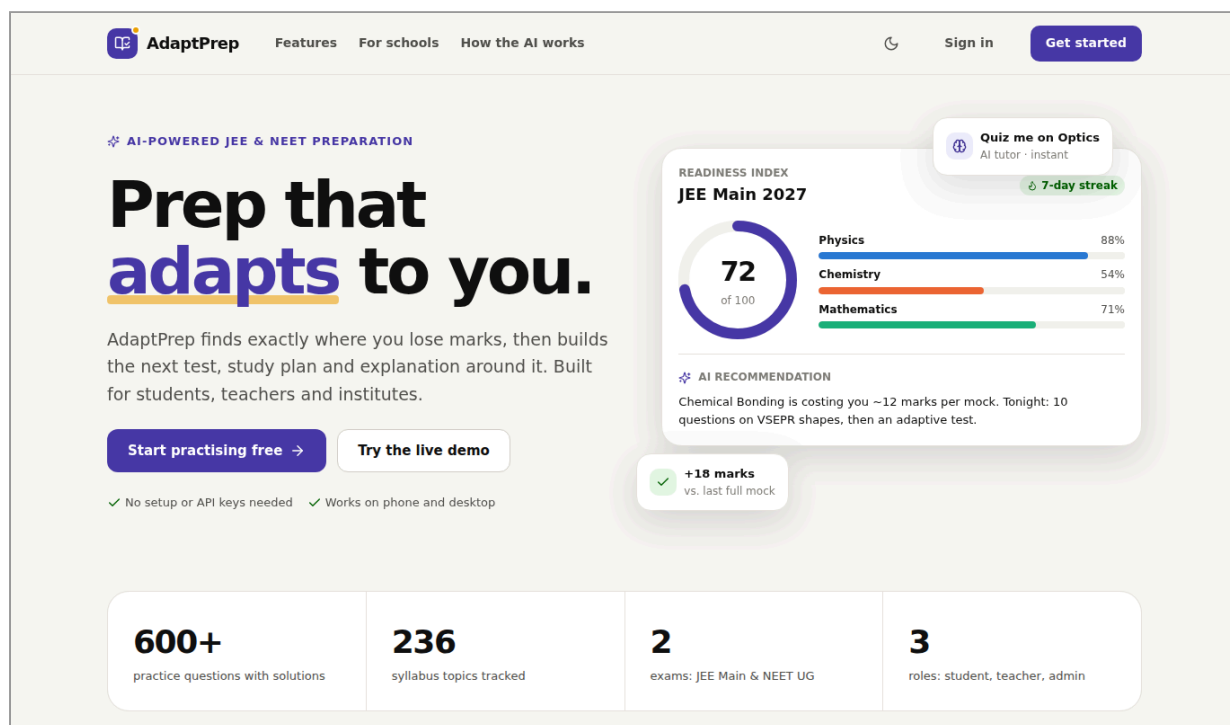


Figure 4.1: Landing page

AdaptPrep

"The fastest way to a better score is knowing exactly which ten topics cost you marks."
AdaptPrep turns every test into a personal plan.

600+ questions · Adaptive AI · JEE & NEET

Welcome back

Sign in to continue your preparation.

Explore instantly with a demo account

Student
Aarav - JEE

Teacher
Priya - 2 classes

Admin
Meera - platform

OR SIGN IN WITH EMAIL

Email
you@school.edu

Password
.....

[Sign in →](#)

Forgot your password? Ask your teacher or administrator to reset it.

New to AdaptPrep? [Create an account](#)

Figure 4.2: Login page with role-based demo accounts

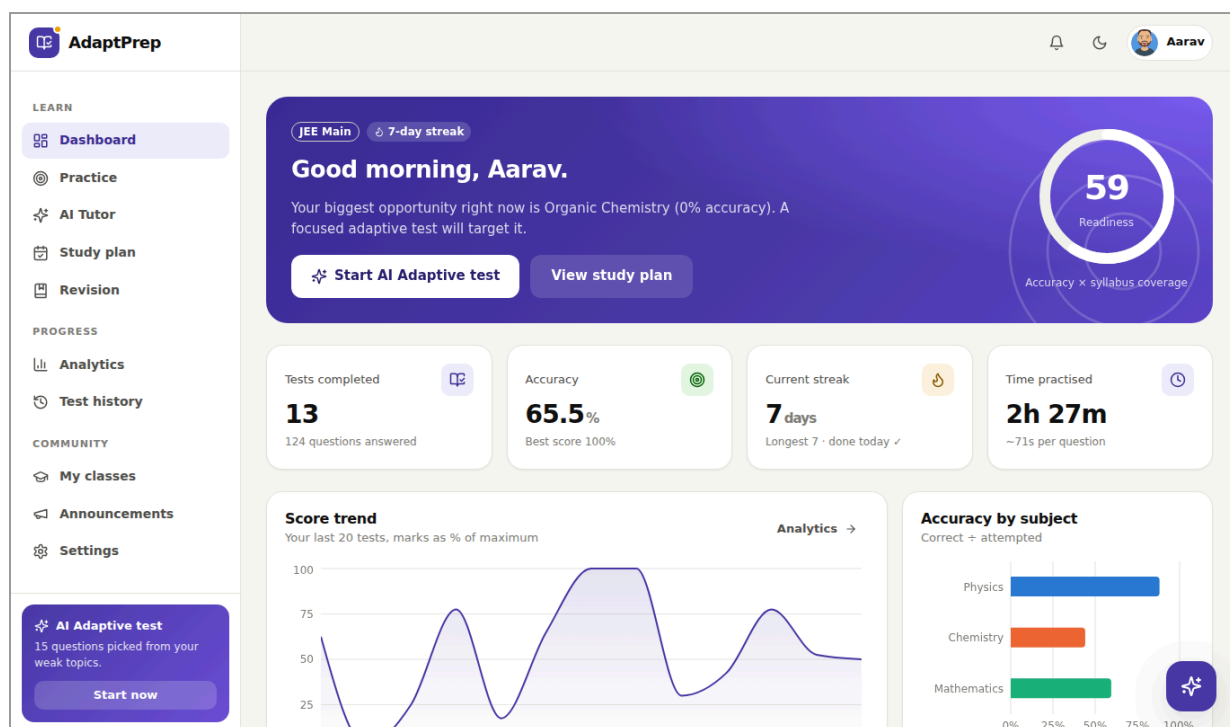


Figure 4.3: Student dashboard: readiness score, accuracy, streak and score trend

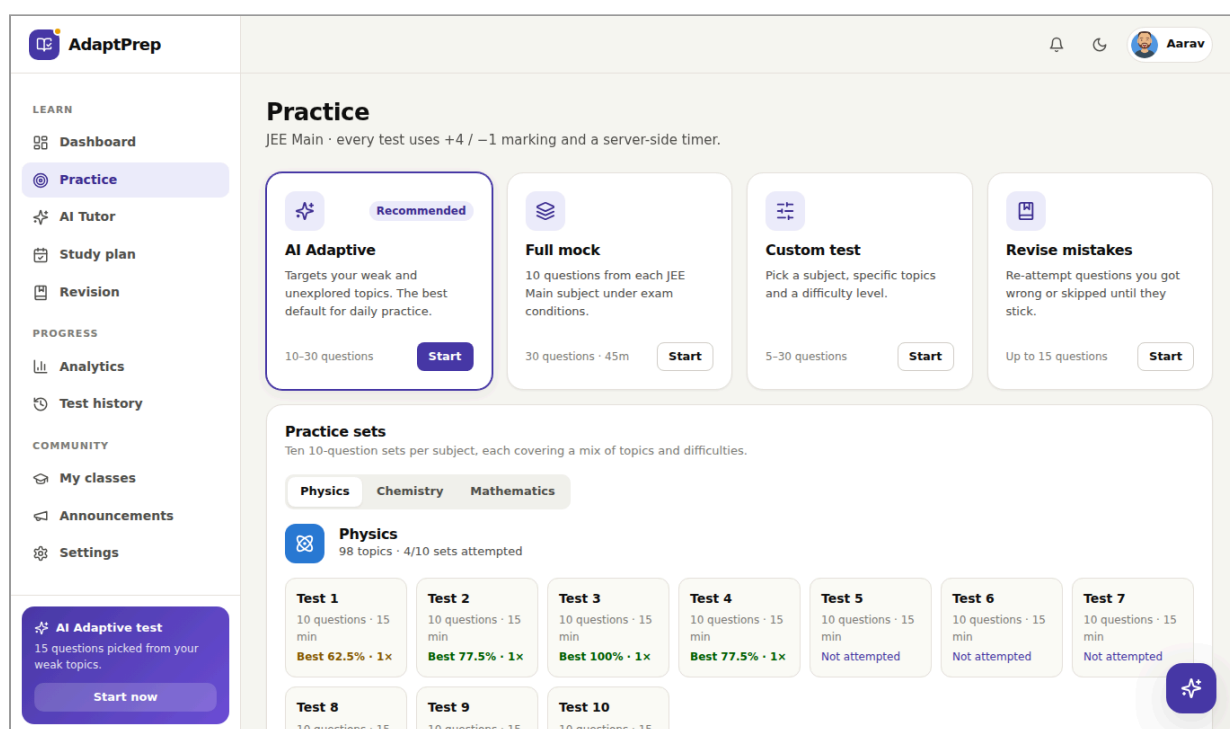


Figure 4.4: Practice page: test sets, AI adaptive test and custom tests

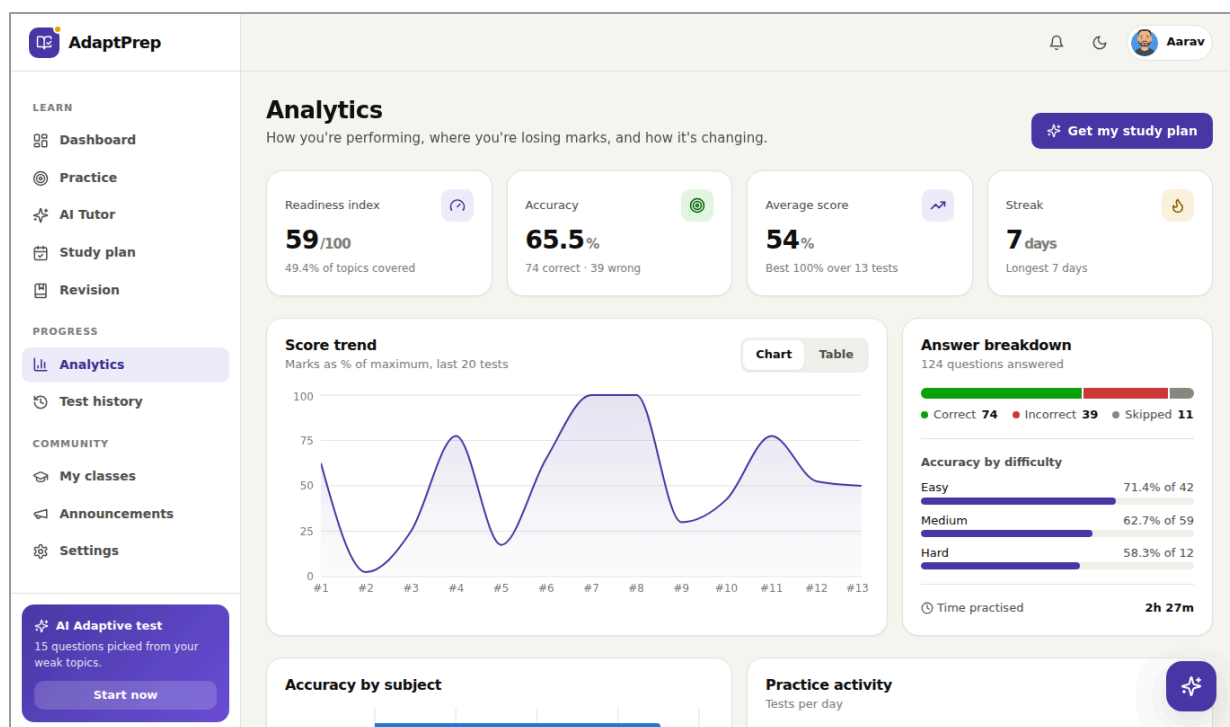


Figure 4.5: Analytics: subject and topic-wise performance

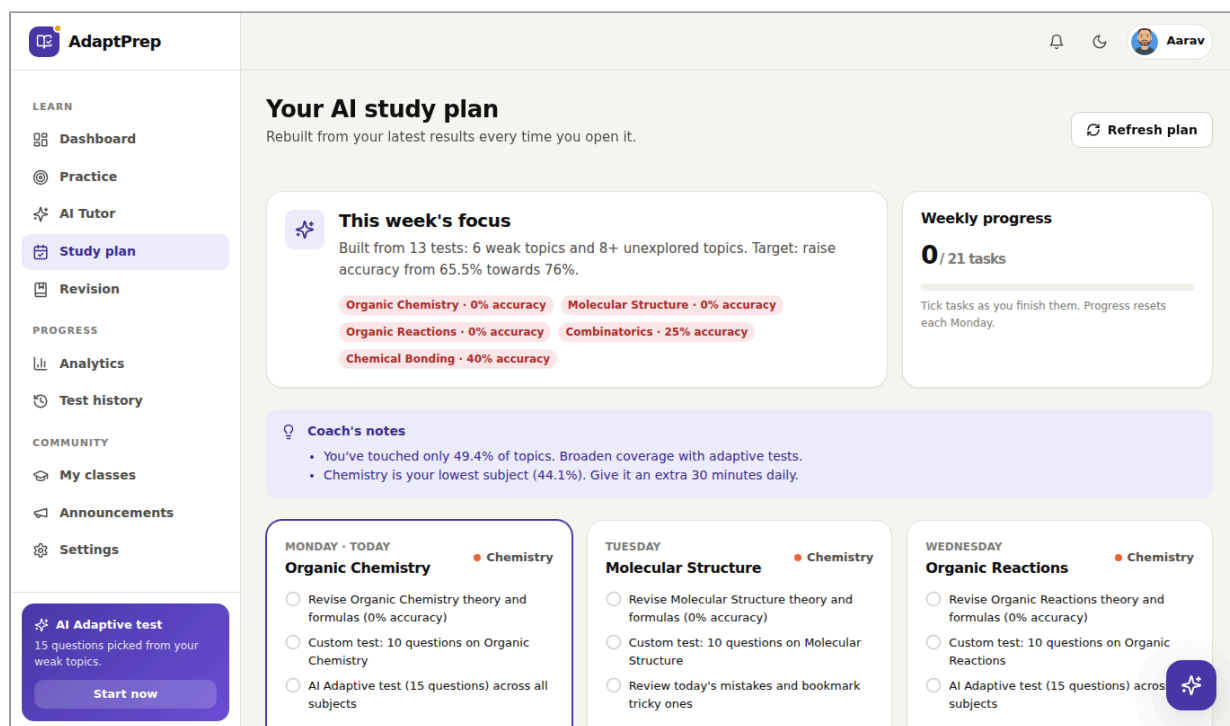


Figure 4.6: Auto-generated weekly study plan

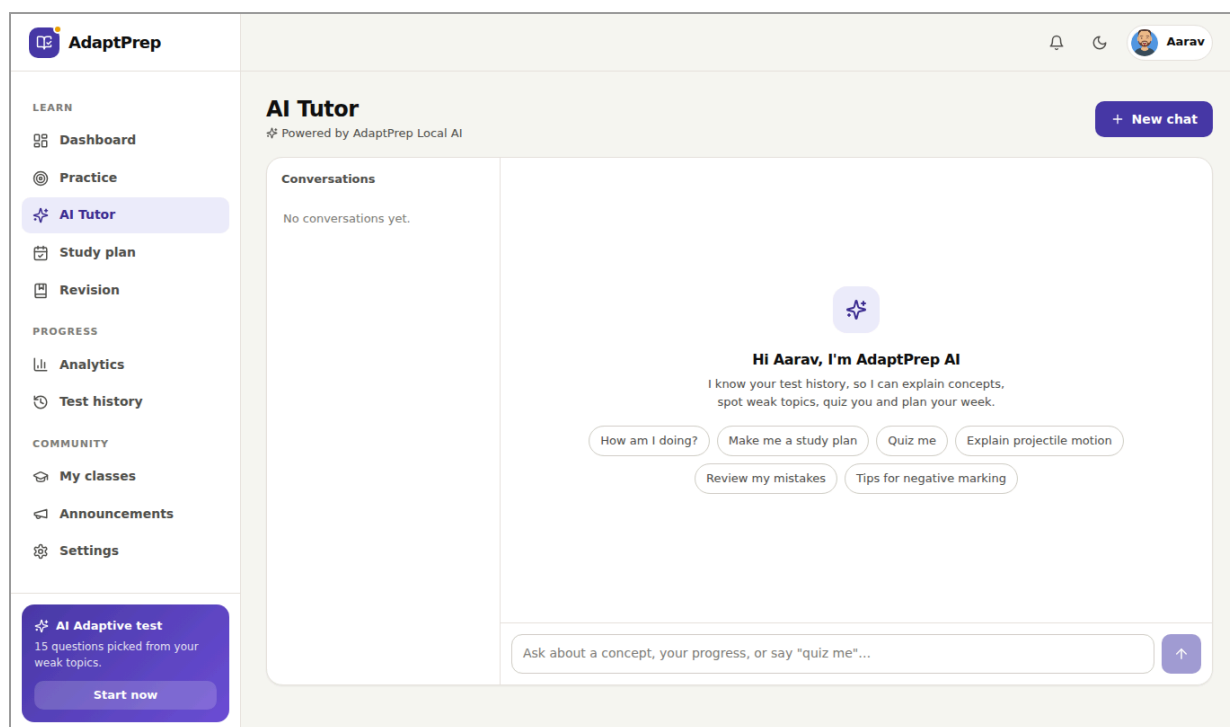


Figure 4.7: AI Tutor

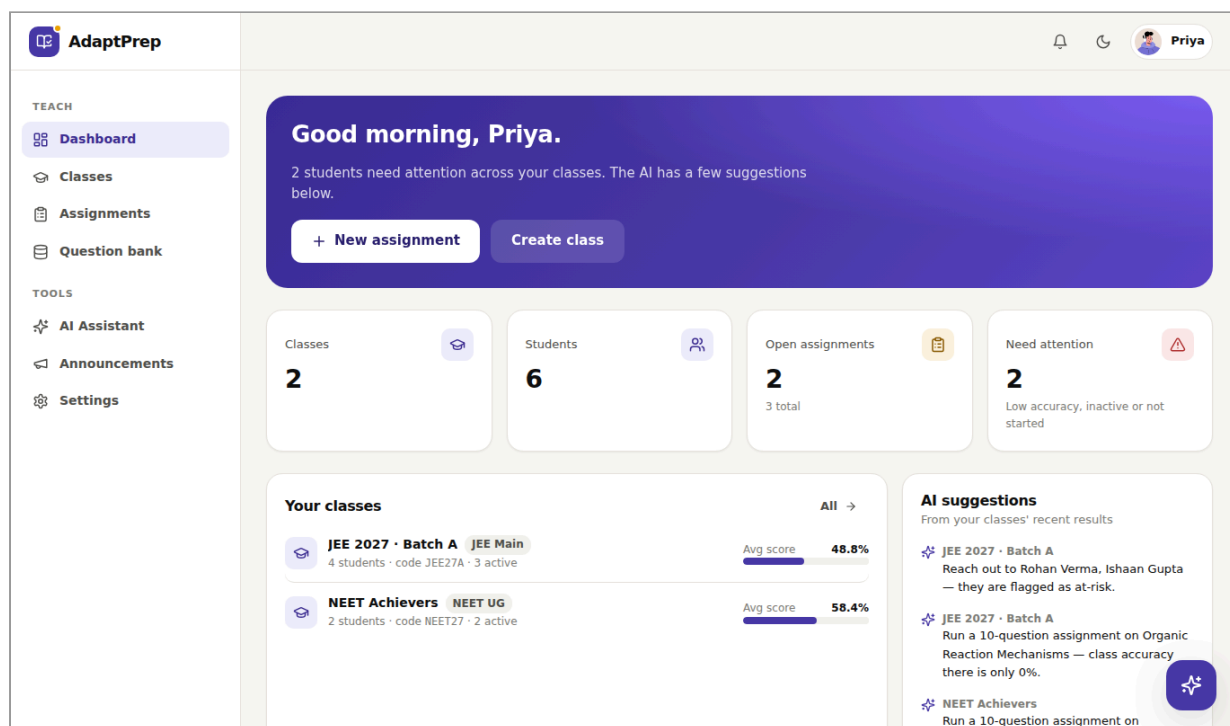


Figure 4.8: Teacher dashboard

AdaptPrep

TEACH

Dashboard

Classes

Assignments

Question bank

TOOLS

AI Assistant

Announcements

Settings

Assignments

Tests you've assigned to your classes.

+ New assignment

Open Closed All

ASSIGNMENT	CLASS	DUE	SUBMISSIONS
Chemical Bonding Checkpoint 8 questions · 12 min	JEE 2027 · Batch A	Oct 2, 7:52 AM in 3 days	1 of 4 25%
Genetics Sprint 10 questions · 15 min	NEET Achievers	Oct 3, 7:52 AM in 4 days	0 of 2 0%

Figure 4.9: Teacher assignments

AdaptPrep

PLATFORM

Overview

Users

Classes

Question bank

Announcements

GOVERNANCE

Audit log

System & AI

AI Assistant

Settings

Users

Manage every account on the platform.

+ Add user

Search name or email

All

Students

Teachers

Admins

Any status

USER	ROLE	STATUS	JOINED	LAST LOGIN
Kabir Singh kabir@adaptprep.dev	student NEET	Active	Sep 28, 2026	never
Ananya Iyer neet@adaptprep.dev	student NEET	Active	Sep 28, 2026	never
Ishaan Gupta ishaan@adaptprep.dev	student JEE	Active	Sep 28, 2026	never
Rohan Verma rohan@adaptprep.dev	student JEE	Active	Sep 28, 2026	never
Diya Patel diya@adaptprep.dev	student JEE	Active	Sep 28, 2026	never
Aarav Sharma student@adaptprep.dev	student JEE	Active	Sep 28, 2026	just now
Priya Mehta teacher@adaptprep.dev	teacher	Active	Sep 28, 2026	just now
Meera Nair admin@adaptprep.dev	admin	Active	Sep 28, 2026	just now

Figure 4.10: Admin: user management

13

Discussion

The adaptive engine behaves as intended. For the demo student, whose Organic Chemistry accuracy is 0%, the dashboard highlights that topic as the biggest opportunity, and adaptive tests draw a larger share of questions from it while still mixing in unseen questions from other topics. Exam-style +4/-1 marking and server-side timers (with automatic submission of expired attempts) make the practice realistic and prevent tampering, because answers are revealed only after submission. The dual AI design keeps every AI feature available even without an API key. Role-based access control and token versioning mean that suspending a user or changing their password immediately invalidates old sessions.

5. Conclusion & Future Scope

Conclusion: AdaptPrep demonstrates how advanced web technologies (a React SPA, a secured Express REST API, JWT authentication, serverless deployment and AI integration) can be combined into a practical learning product. The system personalises JEE and NEET practice with an adaptive selection algorithm, gives students clear analytics and a study plan, provides teachers with class and assignment management, and gives administrators oversight through user management and audit logs. It is secure, tested and deployed to the cloud.

Future scope:

- Item Response Theory (IRT) or knowledge-tracing models to estimate student ability more precisely.
- Larger question banks with images, diagrams and numerical-answer questions.
- A mobile app (React Native) with offline practice and push reminders.
- Live proctored mock tests and all-India rank prediction.
- Regional-language support (Hindi, Marathi) and voice-based tutoring.
- Migration to a relational database (PostgreSQL) for large-scale analytics.

6. References

1. React documentation. <https://react.dev>
2. Vite documentation. <https://vitejs.dev>
3. Express.js documentation. <https://expressjs.com>
4. Node.js documentation. <https://nodejs.org/docs>
5. JSON Web Tokens, RFC 7519. <https://datatracker.ietf.org/doc/html/rfc7519>
6. Recharts documentation. <https://recharts.org>
7. Vercel documentation. <https://vercel.com/docs>
8. Upstash Redis REST API. <https://upstash.com/docs/redis/features/restapi>
9. Google Gemini API documentation. <https://ai.google.dev>
10. P. S. Efraimidis and P. G. Spirakis, "Weighted random sampling with a reservoir," *Information Processing Letters*, vol. 97, no. 5, pp. 181–185, 2006.
11. National Testing Agency, JEE (Main) and NEET (UG) information bulletins. <https://nta.ac.in>